# SQL2Mongo

GitHub Link: https://github.com/71percentbanana/MQL

A SQL-to-MongoDB Query Compiler and Execution Engine

**Phase 3: Final Project Report**

BCSE307L / BCSE307P: Compiler Design and Compiler Design Lab

Submitted by: Alan James
Registration Number: 24BCT0181

School of Computer Science and Engineering, VIT Vellore
Fall Semester 2026–27

October 2026

# Abstract

Most developers learn SQL before they ever work with MongoDB, and moving to MongoDB means relearning how to write the same queries in a very different language. SQL2Mongo addresses this. It is a source-to-source compiler, written in C, that translates SQL `SELECT` queries into MongoDB aggregation pipelines and runs them against document collections stored as JSON files.

The compiler implements seven phases: lexical analysis, recursive-descent parsing with parse-tree and AST output, symbol-table-driven semantic analysis, intermediate code in three-address code, quadruple and triple form, an optimization pass, MongoDB Query Language (MQL) code generation, and execution. Each phase can be shown on its own through a command-line flag, and a local web dashboard presents the same output page by page.

The supported language covers projections, `WHERE` filters with `AND`, `OR` and parentheses, six comparison operators, `ORDER BY` and `LIMIT`. The compiler is 2,456 lines of C in ten modules and uses only the C standard library. Of 21 test queries covering valid, boundary and erroneous input, 19 behave exactly as expected; the remaining 2 expose limitations in the generated target code, which are documented with their fixes. All 9 automated integration tests pass.

# Contents

**Abstract** i

**Phase 3 Overview** iii

**1 Introduction** 1

**2 Problem Statement** 2

**3 Objectives** 3

**4 Background Study** 4

**5 System Design** 6

**6 Methodology** 9

**7 Compiler Design Concepts Used** 10

**8 Implementation** 16

**9 Testing and Results** 20

**10 Conclusion** 26

**11 Future Enhancements** 26

**References** 27

**A User Instructions** 28

**B Project Structure** 29

**C Code Excerpts and Sample Data** 29

# Phase 3 Overview

The objective of Phase 3 is to complete the project, integrate all modules and validate the final system. Phase 1 defined the query language and produced the lexer and parser. Phase 2 added the symbol table, semantic checks, the abstract syntax tree and three-address code. Phase 3 adds the following:

- **Complete intermediate representation.** Quadruples and triples are generated alongside three-address code.
- **Back end.** An optimization pass and a code generator that emits a MongoDB aggregation pipeline for every valid query.
- **Execution on real data files.** An execution engine runs the compiled query against collections stored as JSON files in `data/`, in the same format that `mongoexport --jsonArray` produces. Editing a file changes the results of the next query without rebuilding.
- **Web dashboard.** A local web interface with three pages: Query (SQL in, MongoDB pipeline and documents out), Compiler phases (the output of all seven phases), and Database (the contents of each collection file).
- **Validation.** 21 test queries, including two new execution-error cases, and 9 automated integration tests that run the real executable.

# Chapter 1: Introduction

SQL2Mongo lets someone who already knows SQL write a query in SQL and get the equivalent MongoDB query. It does this by applying the full compiler pipeline to the two languages, with SQL as the source and the MongoDB Query Language (MQL) as the target.

## 1.1 Background

Relational databases are queried with SQL, a declarative language with a fixed keyword grammar. Document databases such as MongoDB store JSON-like documents and are queried with MQL, where a query is itself a JSON structure. MongoDB's aggregation framework expresses a query as a pipeline of stages such as `$match`, `$sort`, `$limit` and `$project` [7].

Both languages describe the same operations, but with different syntax and structure, so moving a query from one to the other is a translation problem.

## 1.2 Motivation

Most students and developers learn SQL first. It is what database courses teach, and it is the language people reach for when they think about querying data. Many of them later work on projects that use MongoDB, and they then have to learn a second, very different way of writing the same queries.

I faced this myself during my internship, where the project used MongoDB. I already knew how to write the queries I needed in SQL, but I had to work out how to express each one in MongoDB: which operator replaces `>=`, how `AND` and `OR` turn into nested `$and` and `$or` arrays, and in what order the pipeline stages must go. A small mistake in this translation does not raise an error. The query simply returns the wrong documents, or none at all.

This project grew out of that experience. Translating one query language into another is exactly what a compiler does, so I built one that takes the SQL a developer already knows and produces the equivalent MongoDB query. Because every phase prints its output, the tool also shows how each part of the SQL query maps to MongoDB, which helps someone who is learning MongoDB as well as someone who only needs the translated query.

## 1.3 Problem Area

The project sits in source-to-source compilation (transpilation) and database query processing. It combines classic front-end phases (scanning, parsing, semantic checks) with query-processing ideas (relational operators, query plans, predicate pushdown).

## 1.4 Summary of the Three Phases

| Phase | Result |
|---|---|
| Phase 1 | Query language and grammar, lexer with token table, recursive-descent parser with a validity check. |
| Phase 2 | Symbol table and semantic checks, abstract syntax tree, three-address code. |
| Phase 3 | Quadruples and triples, optimizer, MQL code generator, execution engine over JSON collection files, web dashboard, and the full test suite. |

**Table 1.1:** What each phase of the project delivered

# Chapter 2: Problem Statement

A developer who knows SQL but has to work with MongoDB must translate every query by hand. The problem this project solves is: given a SQL `SELECT` query over a known schema, produce an equivalent and valid MongoDB aggregation pipeline, or report exactly where and why the query is invalid.

Hand translation from SQL to MQL has three recurring failure points:

- **Operator mapping.** Each SQL comparison has a different MQL spelling (`>=` to `$gte`, `!=` and `<>` to `$ne`), and equality is written without an operator.
- **Logical structure.** Operator precedence must be preserved: `a AND b OR c` means `(a AND b) OR c`, which becomes an `$or` array containing an `$and` array.
- **Schema errors found late.** A misspelled field or a string compared with a number is not rejected by MongoDB. The query simply returns no documents, so the mistake goes unnoticed.

The translator must therefore check queries before generating code, its diagnostics must point to the line and column of the fault, and its output must be verifiable by running the query on real documents.

## 2.1 Requirements

| Type | Requirement |
|---|---|
| Input | A SQL query, either as a `.sql` file path or as an inline string on the command line, or typed into the web dashboard. |
| Data | Collections stored as JSON arrays of flat documents, one file per collection in `data/`. |
| Output | Token table, parse tree, AST, symbol table, TAC, quadruples, triples, optimization report, MQL pipeline and query results, each selectable by a flag. |
| Functional | Reject lexical, syntax and semantic errors with a message giving the line, column and expected construct. |
| Functional | Return exit code 0 on success, 1 on a lexical or syntax error, 2 on a semantic error and 3 when a collection file is missing or invalid, so tests can be automated. |
| Interface | A local web page that shows the SQL, the generated pipeline, the result documents, every compiler phase and the stored collections. |
| System | Any machine with a C compiler; Node.js 20 or newer for the dashboard. No database server or third-party library is needed. |

**Table 2.1:** System requirements

# Chapter 3: Objectives

Each objective set at the start of the project is listed with its status and the place in this report where the evidence can be found.

**Table 3.1:** Objectives, status and evidence

| No. | Objective | Status | Evidence |
|---|---|---|---|
| 1 | Build a lexical analyser that recognises 11 SQL keywords, identifiers, number and string literals, 7 comparison lexemes and 5 delimiters, skips comments, and tracks line and column. | Done | Section 7.2, `sql_lexer.c` |
| 2 | Build a recursive-descent parser that prints a concrete parse tree and builds an abstract syntax tree. | Done | Sections 7.3 and 7.4 |
| 3 | Build a symbol table of collections and typed fields, and use it to check collections, fields and types. | Done | Section 7.5 |
| 4 | Generate intermediate code as three-address code, quadruples and triples. | Done | Section 7.6 |

| No. | Objective | Status | Evidence |
|---|---|---|---|
| 5 | Apply query optimizations: predicate pushdown and projection pruning. | Done | Section 7.7 |
| 6 | Generate target MQL as a `db.<collection>.aggregate([...])` pipeline. | Done | Section 7.8 |
| 7 | Execute the compiled query on documents loaded from JSON collection files. | Done | Sections 7.9 and 8.4 |
| 8 | Provide a web interface that shows the result and every compiler phase. | Done | Section 8.5, Figures 9.3–9.5 |
| 9 | Validate the system with valid, invalid, boundary and complex test queries and automated tests. | Done | Chapter 9 |

## 3.1 Scope

**In scope:** single-collection `SELECT` queries with a column list or `*`; `WHERE` with `=`, `!=`, `<>`, `>`, `>=`, `<`, `<=`, `AND`, `OR` and parentheses; `ORDER BY` on one field, `ASC` or `DESC`; `LIMIT`; case-insensitive keywords; flat JSON documents with string and number values.

**Out of scope:** `JOIN`, `GROUP BY` and aggregate functions, subqueries, `IN`, `LIKE` and `BETWEEN`, data-changing statements (`INSERT`, `UPDATE`, `DELETE`), nested documents and arrays, and a connection to a MongoDB server. `NOT` is recognised by the lexer as a reserved word but is not yet part of the grammar.

# Chapter 4: Background Study

SQL-to-MQL translators already exist, but they hide their internals behind a product or a third-party parser. SQL2Mongo builds every phase by hand so each one can be shown and explained.

## 4.1 Existing Approaches

| Approach | How it works | Difference from SQL2Mongo |
|---|---|---|
| MongoDB SQL to Aggregation Mapping Chart [6] | Official reference table that maps SQL clauses to pipeline stages. | A manual guide; it performs no translation or checking. |
| mongotranslate, MongoDB BI Connector [11] | Command-line tool that shows how a SQL query is expressed as an aggregation pipeline, using a schema (DRDL) file. | Part of a large commercial product; its phases are not visible. |
| sql-to-mongo-db-query-converter [12] | Java library that converts SQL into MongoDB find, update, delete and aggregation queries. | Parsing is delegated to the JSqlParser library, so there is no hand-built lexer or parser. |
| SQL2Mongo (this project) | Hand-written lexer, parser, semantic analyser, IR generator, optimizer, code generator and executor in C. | Every phase prints its own output; schema errors are caught before code generation. |

**Table 4.1:** Comparison with existing SQL-to-MongoDB translators

## 4.2 SQL-to-MQL Mapping

The target forms follow MongoDB's own mapping chart [6].

| SQL construct | MQL stage or operator |
|---|---|
| `FROM t` | `db.t.aggregate([...])` |
| `WHERE` | `$match` |
| `ORDER BY f ASC / DESC` | `$sort: { f: 1 } / { f: -1 }` |
| `LIMIT n` | `$limit: n` |
| `SELECT a, b` | `$project: { _id: 0, a: 1, b: 1 }` |
| `SELECT *` | no `$project` stage |
| `=` | implicit equality `{ f: v }` |
| `!=, <>, >, >=, <, <=` | `$ne`, `$ne`, `$gt`, `$gte`, `$lt`, `$lte` |
| `AND, OR` | `$and: [...]`, `$or: [...]` |

**Table 4.2:** SQL constructs and their MQL equivalents

## 4.3 Compiler Techniques Used

- **Phases of a compiler.** The project follows the analysis–synthesis model of Aho, Lam, Sethi and Ullman [1]: scanning, parsing, semantic analysis, intermediate code, optimization and code generation.

- **Recursive-descent parsing.** One function per non-terminal; operator precedence is encoded by grammar layering (`OR` below `AND` below comparisons).
- **Symbol tables and type checking.** The schema catalog acts as a symbol table holding each field's name and type.
- **Intermediate representations.** Three-address code, quadruples and triples, with relational-algebra operators [2] (scan, filter, sort, limit, project) as the instruction set.
- **Query optimization.** Filtering early (predicate pushdown) is a standard relational technique [3]. MongoDB's pipeline optimization guide [8] also moves `$match` ahead of `$sort` and keeps `$project` last.

## 4.4 The Collection File Format

MongoDB's `mongoexport` tool writes a collection as JSON; with the `--jsonArray` option the output is a single array of documents [10]. SQL2Mongo stores each collection in exactly this form, using the JSON grammar of RFC 8259 [5]. The files can therefore be imported into a real MongoDB server unchanged with `mongoimport --jsonArray`, and the generated pipelines can be run against the same data.

## 4.5 Technology Selection

C was chosen from the course's recommended languages because it forces every data structure to be explicit [4]. The lexer, parser and JSON reader are written by hand rather than with Flex, Bison or a JSON library, so the scanning and parsing algorithms are visible in the source. The dashboard uses Node.js with no third-party packages, because it only needs to start the compiler and serve three static files. Development used GCC (MinGW 6.3.0), Visual Studio Code and Git.

# Chapter 5: System Design

SQL2Mongo is a single-pass pipeline of seven phases. The parser builds one AST, and every later phase reads it. A separate web layer runs the compiler and displays its output.

## 5.1 Architecture

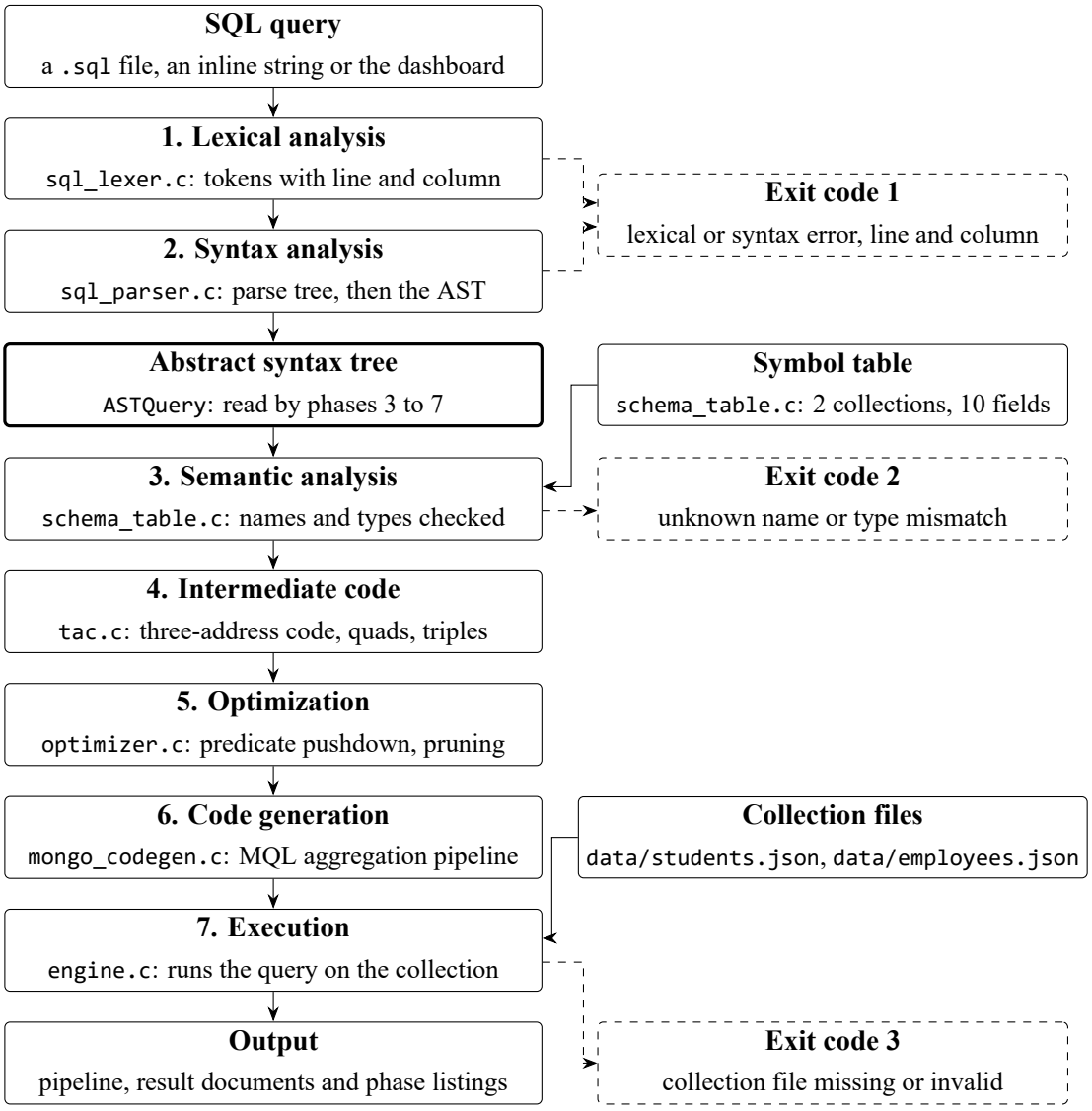



**Figure 5.1:** Architecture of the compiler (dashed lines show error exits)

The phases run in the order set by `main.c`. The symbol table feeds only semantic analysis, and the collection files feed only execution. Each kind of error stops compilation with its own exit code, so later phases never see an invalid query. During parsing, the parser asks the lexer for each next token on demand; the token table printed by `--tokens` comes from a separate display pass.

## 5.2 Module Description

| Module | Files | Responsibility |
|---|---|---|
| Lexical analysis | `sql_lexer.c`, `sql_lexer.h` | Groups characters into tokens and records their positions. |
| Parsing | `sql_parser.c`, `sql_parser.h` | Recursive-descent parser; prints the parse tree. |
| Syntax tree | `sql_ast.c`, `sql_ast.h` | AST node types, constructors, printer and cleanup. |
| Semantic analysis | `schema_table.c`, `schema_table.h` | Symbol table of collections and typed fields; semantic checks. |
| Intermediate code | `tac.c`, `tac.h` | Three-address code, quadruples and triples. |
| Optimization | `optimizer.c`, `optimizer.h` | Optimization pass and report. |
| Query plan | `query_plan.c`, `query_plan.h` | Relational query plan with cost estimates (not yet exposed by a flag). |
| Code generation | `mongo_codegen.c`, `mongo_codegen.h` | MongoDB aggregation pipeline. |
| Execution | `engine.c`, `engine.h` | JSON collection reader and query interpreter. |
| Driver | `main.c` | Command-line flags; runs the phases in order. |
| Web server | `dashboard/server.mjs` | Serves the dashboard and its JSON API. |
| Compiler bridge | `dashboard/compiler.mjs` | Runs the executable and turns its output into structured data. |
| Web page | `dashboard/public/` | Query, Compiler phases and Database pages. |

**Table 5.1:** Modules and their responsibilities

Section 8.2 lists the size and main functions of each C module.

## 5.3 Web Interface Contract

The dashboard server listens on `127.0.0.1:3000`. For each query it starts the compiler with `--all`, so the browser receives every phase from a single run. The server exposes four routes:

| Request | Response |
|---|---|
| `GET /` | The dashboard page, with `app.js` and `styles.css`. |
| `GET /api/health` | Server status and the name, description and output of each of the seven phases. |
| `GET /api/database` | Every collection in `data/`, read from the same files the compiler uses. |
| `POST /api/compile` | Body `{"sql": "..."}`. Returns success, exit code, run time, the diagnostic, the status and output of each phase, tokens, symbol table, pipeline text and stages, result documents, evaluated and returned counts, and the raw compiler output. |

**Table 5.2:** Dashboard API

Phase status is one of *complete*, *error* or *skipped*. The failed phase is found from the exit code and the diagnostic text, and every phase after it is marked as skipped.

# Chapter 6: Methodology

The compiler was built as a single-pass, phase-by-phase pipeline around one shared data structure, the AST, with every phase observable on its own.

1. **Specify the language first.** Tokens, keywords, operators and the grammar (Chapter 7) were fixed before any code was written, so the lexer and parser had an exact target.
2. **One module per phase.** Each phase is a `.c`/`.h` pair with a small public interface, for example `lexer_next_token`, `parser_parse`, `schema_validate_query` and `tac_generate`.
3. **AST as the contract between phases.** The parser produces an `ASTQuery`; every later phase reads it. This keeps phases independent and lets any of them be tested alone.
4. **Fail fast with phase-specific diagnostics.** Compilation stops at the first error. The message names the phase, the line and column, and what was expected. The exit code tells the phase.
5. **Make every phase visible.** Each phase has a print function and a command-line flag, so a reviewer can inspect any intermediate form.
6. **Verify by execution on real data.** The execution engine evaluates the same AST against the collection files. Its results confirm that the generated pipeline means what the SQL query means.
7. **Add the web layer last.** The dashboard calls the finished executable and parses its output, so it cannot disagree with the command-line compiler.
8. **Test against expected results.** Every test query has a known exit code and result; the integration tests check them automatically.

The work was split across the three project phases summarised in Table 1.1.

# Chapter 7: Compiler Design Concepts Used

SQL2Mongo uses a core compiler concept in each phase. The running example in this chapter is the query below (test case 1).

```
SELECT name, age FROM students WHERE age >= 20 ORDER BY age DESC LIMIT 3;
```

## 7.1 Concepts and Where They Are Applied

| Concept | Application in the project | Added in |
|---|---|---|
| Lexical analysis, token patterns | Keywords, identifiers, literals, operators and delimiters, with positions | Phase 1 |
| Context-free grammar | The query language in EBNF | Phase 1 |
| Recursive-descent (LL(1)) parsing | One function per non-terminal; precedence by layering | Phase 1 |
| Parse tree and AST | Concrete tree printed for every query; AST shared by later phases | Phases 1–2 |
| Symbol table, scope | Collections and typed fields in one global scope | Phase 2 |
| Type checking | Numeric fields compared only with numbers, strings only with strings | Phase 2 |
| Three-address code | Post-order translation of the filter tree | Phase 2 |
| Quadruples and triples | Two table forms of the same intermediate code | Phase 3 |
| Optimization | Predicate pushdown and projection pruning in the pipeline | Phase 3 |
| Target code generation | Recursive emission of the MQL pipeline | Phase 3 |
| Interpretation | Tree-walking execution on the collection files | Phase 3 |
| Error detection | Stop at the first error with a position and an exit code per phase | Phases 1–3 |

**Table 7.1:** Compiler concepts and where they are used

## 7.2 Lexical Analysis

The scanner (`sql_lexer.c`) turns the character stream into tokens using maximal munch with one character of lookahead. The lookahead (`peek_next`) separates `>` from `>=`, `<` from `<=` and `<>`, and `-` from a `--` comment. Each token records its type, lexeme, numeric value, line and column.

| Token class | Pattern or lexemes | Token type |
|---|---|---|
| Keywords (11, case-insensitive) | `SELECT FROM WHERE ORDER BY ASC DESC LIMIT AND OR NOT` | `TOK_SELECT` … `TOK_NOT` |
| Identifier | `[A-Za-z_][A-Za-z0-9_]*`, not a keyword | `TOK_IDENT` |
| Number literal | `[0-9]+(\.[0-9]*)?` | `TOK_NUMBER` |
| String literal | `'…'` or `"…"`, with `\` escapes | `TOK_STRING` |
| Comparison operator | `=  !=  <>  >  >=  <  <=` | `TOK_EQ`, `TOK_NEQ`, `TOK_GT`, `TOK_GTE`, `TOK_LT`, `TOK_LTE` |
| Delimiter | `,  ;  *  (  )` | `TOK_COMMA` … `TOK_RPAREN` |
| Comment (discarded) | `--` … to end of line, `/*` … `*/` | none |

**Table 7.2:** Token classes recognised by the lexer

Keywords are recognised after an identifier is scanned: the lexeme is upper-cased and compared with the keyword list (`check_keyword`). Three inputs raise a lexical error: an unknown character such as `@`, a lone `!`, and an unterminated string. Figure 7.1 shows the token table for the running example.

```
PS C:\Users\alanv\compiler-project> .\sql2mongo.exe "SELECT name, age FROM students WHERE age >= 20 ORDER BY age DESC LIMIT 3;" --tokens
======================================================================
  STAGE 1: Lexical Analysis (Scanner)
======================================================================
+----+----------------------+--------------------+------+------+
| #  | Token Type           | Lexeme             | Line | Col  |
+----+----------------------+--------------------+------+------+
| 1  | KEYWORD(SELECT)      | SELECT             | 1    | 1    |
| 2  | IDENTIFIER           | name               | 1    | 8    |
| 3  | COMMA(,)             | ,                  | 1    | 12   |
| 4  | IDENTIFIER           | age                | 1    | 14   |
| 5  | KEYWORD(FROM)        | FROM               | 1    | 18   |
| 6  | IDENTIFIER           | students           | 1    | 23   |
| 7  | KEYWORD(WHERE)       | WHERE              | 1    | 32   |
| 8  | IDENTIFIER           | age                | 1    | 38   |
| 9  | OPERATOR(>=)         | >=                 | 1    | 42   |
| 10 | NUMBER               | 20                 | 1    | 45   |
| 11 | KEYWORD(ORDER)       | ORDER              | 1    | 48   |
| 12 | KEYWORD(BY)          | BY                 | 1    | 54   |
| 13 | IDENTIFIER           | age                | 1    | 57   |
| 14 | KEYWORD(DESC)        | DESC               | 1    | 61   |
| 15 | KEYWORD(LIMIT)       | LIMIT              | 1    | 66   |
| 16 | NUMBER               | 3                  | 1    | 72   |
| 17 | SEMICOLON(;)         | ;                  | 1    | 73   |
+----+----------------------+--------------------+------+------+
Total Tokens Scanned: 17 | Lexical Errors: 0 | Status: PASS

======================================================================
  FINAL VERIFICATION: ALL 7 COMPILER PHASES EXECUTED SUCCESSFULLY
======================================================================
```

**Figure 7.1:** Token table printed by `sql2mongo.exe --tokens` for test case 1

## 7.3 Syntax Analysis

The parser (`sql_parser.c`) is a hand-written recursive-descent (LL(1)) parser: one function per non-terminal, deciding each step from the current token only. The grammar in EBNF is:

```
query         → select_clause from_clause [where_clause]
                [order_clause] [limit_clause] [";"] EOF
select_clause → "SELECT" ( "*" | IDENT { "," IDENT } )
from_clause   → "FROM" IDENT
```

```
where_clause  → "WHERE" or_expr
or_expr       → and_expr { "OR" and_expr }
and_expr      → primary { "AND" primary }
primary       → "(" or_expr ")" | IDENT comp_op literal
comp_op       → "=" | "!=" | "<>" | ">" | ">=" | "<" | "<="
literal       → NUMBER | STRING
order_clause  → "ORDER" "BY" IDENT [ "ASC" | "DESC" ]
limit_clause  → "LIMIT" NUMBER
```

Precedence comes from the layering: `primary` binds tightest, then `AND`, then `OR`. The `{ … }` repetitions are written as `while` loops that fold operands to the left, so `a AND b AND c` becomes `(a AND b) AND c`. With the `--tree` flag the parser prints the concrete parse tree, which records every rule and terminal matched; the WHERE part of the running example is:

```
<WhereClause>
 ├── Terminal: 'WHERE'
 └── <ConditionExpression>
      └── <ComparisonExpr>
          ├── <FieldIdentifier: "age">
          ├── <ComparisonOp: ">=">
          └── <IntegerLiteral: 20>
```

On a syntax error the parser records the first error only and reports the expected construct, for example:

```
Syntax error at [Line 1, Col 23]: Expected collection/table name identifier,
but found 'WHERE' (KEYWORD(WHERE))
```

## 7.4 Abstract Syntax Tree

The AST (`sql_ast.h`) drops punctuation and keeps meaning. The root `ASTQuery` holds the collection name, the projection list (or a wildcard flag), an optional `WHERE` tree, and the `ORDER BY` and `LIMIT` settings. The `WHERE` tree is built from `ASTFilter` nodes of two kinds:

- **Binary node:** a field, a comparison operator and a typed literal (`INT`, `FLOAT` or `STRING`).
- **Logical node:** `AND` or `OR` with left and right children.

Filter nodes are allocated on the heap and freed recursively by `ast_free` after compilation. Figure 7.2 shows the AST for a query with a parenthesised `OR` inside an `AND`: the `OR` node is the left child of the `AND` node, exactly as the grammar's precedence requires.

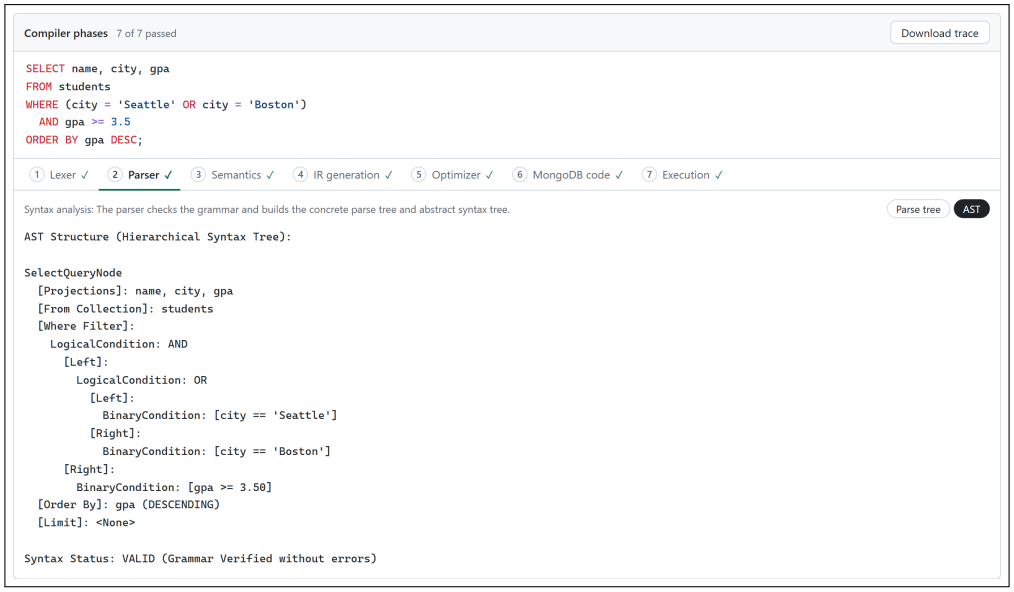



**Figure 7.2:** AST for a nested `AND`/`OR` query, shown on the Compiler phases page

## 7.5 Symbol Table and Semantic Analysis

The schema catalog (`schema_table.c`) is the symbol table. Each entry is a field symbol scoped to its collection, with a data type and storage width. It holds 2 collections and 10 symbols.

| Collection | Field | Type | Width |
|---|---|---|---|
| students | name, city, status | STRING | 64 B |
| students | age | INT | 4 B |
| students | gpa | FLOAT | 8 B |
| employees | name, department, role | STRING | 64 B |
| employees | experience | INT | 4 B |
| employees | salary | FLOAT | 8 B |

**Table 7.3:** Symbol table (schema catalog)

Lookups are case-insensitive. The semantic analyser walks the AST and applies four checks in order, stopping at the first failure:

1. The collection in `FROM` exists.
2. Every column in `SELECT` exists in that collection.
3. Every field in `WHERE` exists, and its type matches the literal: numeric fields take number literals, string fields take string literals.
4. The `ORDER BY` field exists.

For example, `WHERE age = 'twenty'` fails with `Semantic Type Error: Cannot compare numeric field 'age' with string literal 'twenty'`. The symbol table is also used by the execution engine to give each number read from a collection file its declared type.

## 7.6 Intermediate Code Generation

The IR generator (`tac.c`) produces three-address code, then derives quadruples and triples from it. It works in two steps:

1. **Condition code.** A post-order walk of the `WHERE` tree gives each comparison a new temporary, then combines child temporaries at each `AND` or `OR` node.
2. **Operator chain.** Relational operators are chained in a fixed order, `SCAN` → `FILTER` → `SORT` → `LIMIT` → `PROJECT`, each consuming the previous temporary. Absent clauses are skipped.

For test case 2 (`SELECT name, department, salary FROM employees WHERE salary >= 70000 AND department = 'Engineering' ORDER BY salary DESC`) the output is:

```
(1)  t1 = SCAN(employees)
(2)  t2 = salary >= 70000
(3)  t3 = department == 'Engineering'
(4)  t4 = t2 AND t3
(5)  t5 = FILTER(t1, t4)
(6)  t6 = SORT(t5, salary (DESC))
(7)  t7 = PROJECT(t6, [name,department,salary])
```

Quadruples store the result temporary explicitly (Figure 7.3). Triples drop it, and later instructions refer to an earlier result by its index: instruction 4 becomes `AND (2) (3)` and instruction 5 becomes `FILTER (1) (4)`.

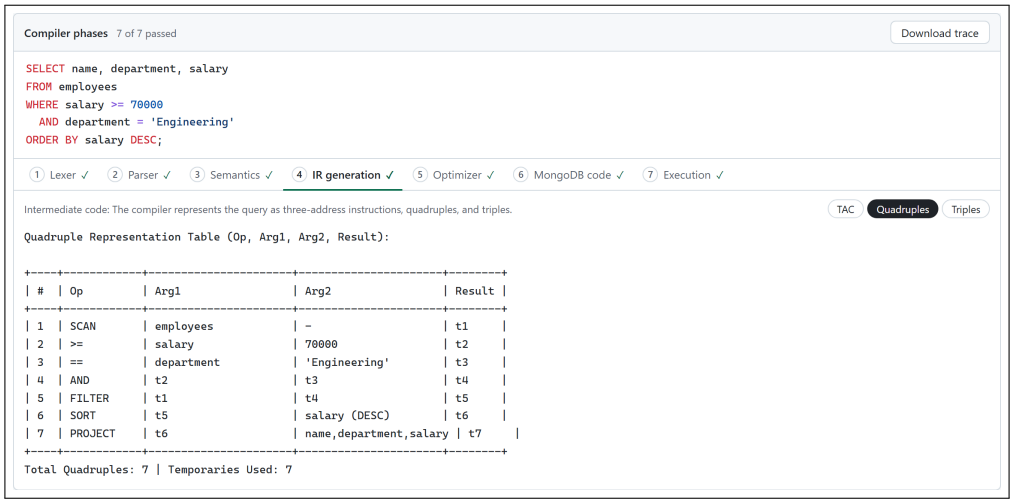



**Figure 7.3:** Quadruples for test case 2 on the Compiler phases page

## 7.7 Code Optimization

The optimizer (`optimizer.c`) runs after IR generation and reports which optimizations apply to the query. Two are built into the shape of the IR and of the generated pipeline:

- **Predicate pushdown.** `FILTER` always comes straight after `SCAN`, so `$match` is always the first pipeline stage. Documents are discarded before the $O(N \log N)$ sort instead of after it.

- **Projection pruning.** A `$project` stage is emitted only when named columns are requested, and it is placed last, as MongoDB's optimization guide recommends [8]. For `SELECT *` the stage is omitted altogether because it would keep every field.

Literal types (`INT`, `FLOAT`, `STRING`) are resolved and checked at compile time, so no type conversion is needed at run time. The grammar has no arithmetic expressions yet, so constant folding currently has nothing to fold; the folding and dead-stage lines of the report are printed for every query.

```
1. [Predicate Pushdown]       : ENABLED & APPLIED (Filter executed before sort/projection)
2. [Projection Column Pruning]: APPLIED (2 attributes selected)
3. [Compile-Time Folding]     : APPLIED (Static type and literal resolution)
4. [Dead Stage Elimination]   : 0 redundant stages detected
```

## 7.8 Target Code Generation

The code generator (`mongo_codegen.c`) walks the AST and emits a `db.<collection>.aggregate([...])` call. Stages appear in the order `$match`, `$sort`, `$limit`, `$project`, and only for clauses present in the query.

Filters are emitted recursively. A comparison becomes `{ field: value }` for equality or `{ field: { $op: value } }` otherwise. A logical node becomes `{ $and: [left, right] }` or `{ $or: [left, right] }`. Because the AST already encodes precedence, nested conditions come out correctly:

```
SELECT name FROM students
WHERE (age > 19 AND gpa >= 3.5) OR status = 'Graduated'
ORDER BY gpa DESC;
```

```
db.students.aggregate([
  { "$match": { "$or": [
      { "$and": [ { "age": { "$gt": 19 } }, { "gpa": { "$gte": 3.50 } } ] },
      { "status": "Graduated" }
  ] } },
  { "$sort": { "gpa": -1 } },
  { "$project": { "_id": 0, "name": 1 } }
]);
```

## 7.9 Interpretation and Execution

The execution engine (`engine.c`) is a tree-walking interpreter over the AST. It reads `data/<collection>.json` (Section 8.4), evaluates the `WHERE` tree recursively for each document, sorts the matches with `qsort` and a field comparator, applies the limit and prints the projected fields as JSON. The query above returns Diana Prince (GPA 3.95), Alice Walker (3.85) and Evan Wright (Graduated, 3.10), which is the correct answer for the data in Appendix C.2.

## 7.10 Error Detection and Recovery

Errors are caught in the earliest phase that can see them. The compiler uses a stop-at-first-error strategy: it reports the fault with its position and halts with a phase-specific exit code.

| Phase | Error | Example input | Message | Exit |
|---|---|---|---|---|
| Lexical | Unknown character | `WHERE age @ 20` | `Lexical error at [Line 1, Col 42]: Unrecognized or malformed token '@'` | 1 |
| Lexical | Unterminated string | `WHERE city = 'Seattle;` | `Lexical error at [Line 1, Col 40]: … malformed token 'Seattle;'` | 1 |
| Syntax | Missing table name | `FROM WHERE age >= 20` | `Expected collection/table name identifier, but found 'WHERE'` | 1 |
| Syntax | Unbalanced parenthesis | `WHERE (age > 19;` | `Expected closing parenthesis ')', but found ';'` | 1 |
| Semantic | Unknown collection | `FROM teachers` | `Collection 'teachers' does not exist in schema catalog` | 2 |
| Semantic | Unknown column | `SELECT name, nonexistent_column` | `Column 'nonexistent_column' in SELECT clause is not defined` | 2 |
| Semantic | Type mismatch | `WHERE city = 25` | `Cannot compare string field 'city' with numeric literal` | 2 |
| Execution | Missing file | `FROM employees`, file absent | `Cannot open collection file 'data/employees.json'` | 3 |
| Execution | Invalid JSON | trailing comma in a document | `Invalid JSON in 'data/students.json': expected '"' but found '}'` | 3 |

**Table 7.4:** Error detection by phase

Figure 7.4 shows three of these diagnostics and the exit code of each run.

```
PS C:\Users\alanv\compiler-project> .\sql2mongo.exe "SELECT @ FROM students;" > $null; $LASTEXITCODE
Lexical error at [Line 1, Col 8]: Unrecognized or malformed token '@'
1
PS C:\Users\alanv\compiler-project> .\sql2mongo.exe "SELECT name students;" > $null; $LASTEXITCODE
Syntax error at [Line 1, Col 13]: Expected keyword 'FROM', but found 'students' (IDENTIFIER)
1
PS C:\Users\alanv\compiler-project> .\sql2mongo.exe "SELECT name FROM students WHERE age = 'twenty';" > $null; $LASTEXITCODE
Semantic Type Error: Cannot compare numeric field 'age' with string literal 'twenty'
2
```

**Figure 7.4:** Lexical, syntax and semantic diagnostics with exit codes 1, 1 and 2

# Chapter 8: Implementation

SQL2Mongo is 2,456 lines of C in ten modules, using only the C standard library. It builds with GCC under `-Wall -Wextra` with no warnings. The dashboard adds about 740 lines of JavaScript, HTML and CSS with no third-party packages.

## 8.1 Languages and Tools

| Item | Choice |
|---|---|
| Compiler language | C, built with `-std=c11` |
| C compiler | GCC 6.3.0 (MinGW) on Windows 11 |
| C libraries | Standard library only (`stdio`, `stdlib`, `string`, `ctype`, `stdbool`) |
| Dashboard | Node.js 20 or newer, built-in modules only; HTML, CSS and JavaScript |
| Tests | Node.js built-in test runner (`node --test`) |
| Editor and version control | Visual Studio Code, Git |

**Table 8.1:** Implementation tools

## 8.2 Modules

| Module | Lines | Responsibility | Key functions |
|---|---|---|---|
| `sql_lexer` | 372 | Scanner, token types, token table | `lexer_next_token`, `check_keyword`, `lexer_print_all` |
| `sql_parser` | 395 | Recursive-descent parser, parse-tree printer | `parser_parse`, `parse_or_expr`, `parse_and_expr`, `parse_primary_filter` |
| `sql_ast` | 223 | AST node types, constructors, printer, cleanup | `ast_create_binary_filter`, `ast_create_logical_filter`, `ast_free` |
| `schema_table` | 236 | Symbol table and semantic checks | `schema_init`, `schema_get_field_type`, `schema_validate_query` |
| `tac` | 257 | Three-address code, quadruples, triples | `tac_generate`, `quads_print`, `triples_print` |
| `optimizer` | 59 | Optimization pass and report | `optimizer_run`, `optimizer_print_report` |
| `query_plan` | 129 | Relational query plan with cost estimates | `plan_generate`, `plan_print` |
| `mongo_codegen` | 137 | MQL aggregation-pipeline generator | `mongo_codegen_pipeline`, `emit_filter_json` |
| `engine` | 381 | JSON collection reader and query interpreter | `load_collection`, `json_read_document`, `eval_filter`, `engine_execute_query` |
| `main` | 267 | Command-line driver that runs the phases in order | `main`, `read_entire_file`, `resolve_data_dir` |
| **Total** | **2,456** | | |

**Table 8.2:** Source modules (lines counted over the `.c` and `.h` files)

## 8.3 Key Algorithms

- **Scanning:** a character-class switch with one-character lookahead and maximal munch; keywords found by upper-casing the identifier and comparing it with the keyword list. Linear in input length.
- **Parsing:** LL(1) recursive descent with one token of lookahead; left-associative operator chains built by loops. Linear in the number of tokens.
- **Semantic analysis:** recursive AST walk with linear symbol lookup in the catalog (10 symbols).
- **IR generation:** post-order traversal of the filter tree; a counter supplies fresh temporaries `t1, t2, ...`. Triples are derived by replacing each temporary with the index of the quadruple that produced it.
- **Code generation:** recursive emission of nested JSON for `$and` and `$or`.
- **Execution:** one pass over the collection file to read it, per-document recursive evaluation of the filter tree, `qsort` for `ORDER BY`, then limit and projection.

## 8.4 Reading Collection Files

The engine contains a small recursive-descent reader for the collection format: an array of flat objects whose values are strings or numbers. It is a second, much smaller parser in the same style as the SQL parser, with one function per construct (`json_read_string`, `json_read_document`, `load_collection`).

- **Locating the file.** The collection name is lower-cased and the file `data/<name>.json` is opened. `main.c` looks for `data/` in the working directory first and then next to the executable, so the compiler works from any folder.
- **Typing values.** A string becomes `TYPE_STRING`. A number takes the type declared in the symbol table, so `95000` and `95000.0` are both read as `FLOAT` for `salary`. A number in an undeclared field is `INT`, or `FLOAT` if it has a decimal point or exponent. `true`, `false` and `null` are skipped.
- **Strings.** The escapes `\"`, `\\`, `\/`, `\n`, `\t` and `\r` are decoded when reading and encoded again when printing results, so the output is always valid JSON.
- **Errors.** A missing file or a syntax error in the file is reported with the file name and what was expected, and the compiler exits with code 3.

The file is read each time a query runs, so editing a collection changes the next result without rebuilding.

## 8.5 Web Dashboard

The dashboard is a Node.js server (`server.mjs`), a bridge to the compiler (`compiler.mjs`) and a single web page with three views.

- **Running the compiler.** For each query the bridge writes the SQL to its own temporary file, starts `sql2mongo.exe` with `--all` without a shell, and waits for it to finish.
- **Parsing the output.** The output is split at the `STAGE n` banners. Tokens and symbol-table rows are read from their text tables, the pipeline is read from the `db.…;` block, and each result document is parsed as JSON. The exit code and diagnostic decide which phase failed.
- **Query page.** SQL editor with syntax highlighting and seven example queries, the generated pipeline, and the result documents as JSON or a table (Figure 9.3).
- **Compiler phases page.** One tab per phase, marked as passed, failed or not reached, with sub-tabs for the parse tree and AST and for TAC, quadruples and triples (Figure 9.4).
- **Database page.** The documents in each collection file with the field types from the symbol table (Figure 9.5).

## 8.6 Limits and Safety

| Limit | Value | Behaviour when exceeded |
|---|---|---|
| Documents per collection | 32 | Exit code 3 with a message |
| Fields per document | 16 | Extra fields are ignored |
| String value length | 127 characters | The value is truncated |
| IR instructions | 32 | Further instructions are not generated |
| Query length (dashboard) | 4,000 characters | Rejected with HTTP 400 |
| Request body (dashboard) | 16 KB | Rejected with HTTP 413 |
| Compiler run time (dashboard) | 6 seconds | Process stopped; HTTP 504 |
| Simultaneous compilations | 4 | Rejected with HTTP 429 |

**Table 8.3:** Limits and their behaviour

The server listens on `127.0.0.1` only, rejects requests from other origins (HTTP 403) and serves only its own four files. The input is always treated as SQL text and never as a file path.

## 8.7 Problems Found and Fixed During Phase 3

1. **The data files were not used.** The execution engine held its documents in C code and ignored `data/*.json`, so editing a file had no effect. The JSON reader of Section 8.4 replaced the built-in records.
2. **Result strings were not escaped.** A quotation mark inside a value would have produced invalid JSON output. Strings are now escaped when printed.

3. **Execution errors were reported as success.** The result of the execution engine was ignored, so a missing collection still ended with `ALL 7 COMPILER PHASES EXECUTED SUCCESSFULLY`. The compiler now stops with exit code 3.
4. **The data folder depended on the working directory.** Running the compiler from another folder could not find the collections. `resolve_data_dir` now also looks next to the executable.

# Chapter 9: Testing and Results

## 9.1 Test Strategy

Tests cover valid, boundary, lexical-error, syntax-error, semantic-error and execution-error inputs. Each case has a known correct exit code and, where it succeeds, a known set of result documents that can be checked by hand against Appendix C.2. Nine integration tests check the most important cases and the dashboard API automatically. All tests were run on 28 October 2026 with a fresh build of the current source.

## 9.2 How the Tests Are Run

| Command | Result |
|---|---|
| `npm run build:compiler` (in `dashboard/`) | Rebuilds `sql2mongo.exe` from `src/*.c`; no warnings |
| `npm test` (in `dashboard/`) | The 9 integration tests of Table 9.3 |
| `sql2mongo.exe "<query>" --all` | Test cases 1–19, compared with the expected exit code and output |
| Compiler run from a folder whose `data/` has a missing or broken file | Test cases 20–21 |

**Table 9.1:** How the tests are run

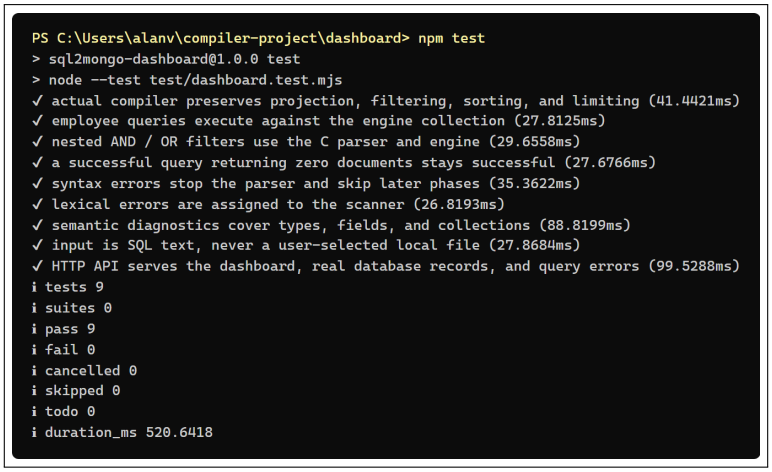



**Figure 9.1:** Automated test run: 9 of 9 integration tests pass

## 9.3 Test Cases and Results

**Table 9.2:** Test cases and results

| # | Category | Input (query or clause) | Expected | Actual | Status |
|---|---|---|---|---|---|
| 1 | Valid | `SELECT name, age FROM students WHERE age >= 20 ORDER BY age DESC LIMIT 3;` | Exit 0; 3 documents, oldest first | Diana (23), Alice (22), Bob (21) | Pass |
| 2 | Valid, `AND` | `SELECT name, department, salary FROM employees WHERE salary >= 70000 AND department = 'Engineering' ORDER BY salary DESC;` | Exit 0; `$and` filter | Sarah Connor (95000), John Doe (72000) | Pass |
| 3 | Valid, * and float | `SELECT * FROM students WHERE gpa >= 3.85;` | Exit 0; no `$project` stage | 3 full documents; pipeline has `$match` only | Pass |
| 4 | Valid, `OR` | `SELECT name, city FROM students WHERE city = 'Seattle' OR city = 'Boston' ORDER BY name ASC;` | Exit 0; `$or` filter, sorted by name | Alice, Bob, Charlie | Pass |
| 5 | Complex | `WHERE (age > 19 AND gpa >= 3.5) OR status = 'Graduated' ORDER BY gpa DESC` | `$and` nested inside `$or` | Correct nesting; Diana, Alice, Evan | Pass |
| 6 | Boundary | `WHERE age > 100` | Exit 0; empty result | 0 documents returned | Pass |
| 7 | Boundary | `select NAME, Age from STUDENTS where AGE < 21 limit 1` (mixed case, no `;`) | Exit 0; 1 document | Charlie Brown returned; MQL keeps the user's casing | Partial |
| 8 | Boundary | `LIMIT 0` | Rejected, or valid MQL | Accepted; emits `$limit: 0` | Partial |

| # | Category | Input (query or clause) | Expected | Actual | Status |
|---|---|---|---|---|---|
| 9 | Lexical error | `WHERE age @ 20` | Exit 1 | Exit 1, unrecognised token `'@'` at col 42 | Pass |
| 10 | Lexical error | `WHERE city = 'Seattle;` | Exit 1 | Exit 1, unterminated string reported | Pass |
| 11 | Syntax error | `SELECT name, age FROM WHERE age >= 20;` | Exit 1 | Exit 1, expected table name, at col 23 | Pass |
| 12 | Syntax error | `ORDER age` | Exit 1 | Exit 1, expected `BY` after `ORDER` | Pass |
| 13 | Syntax error | `LIMIT 2 extra;` | Exit 1 | Exit 1, unexpected trailing tokens | Pass |
| 14 | Syntax error | `WHERE (age > 19;` | Exit 1 | Exit 1, expected closing parenthesis | Pass |
| 15 | Semantic error | `SELECT name, nonexistent_column FROM students WHERE age = 'twenty';` | Exit 2 | Exit 2, unknown column reported first | Pass |
| 16 | Semantic error | `WHERE age = 'twenty'` | Exit 2 | Exit 2, numeric field compared with string | Pass |
| 17 | Semantic error | `WHERE city = 25` | Exit 2 | Exit 2, string field compared with number | Pass |
| 18 | Semantic error | `FROM teachers` | Exit 2 | Exit 2, collection does not exist | Pass |
| 19 | Semantic error | `FROM students ORDER BY salary` | Exit 2 | Exit 2, `ORDER BY` field does not exist | Pass |
| 20 | Execution error | `SELECT name FROM employees;` with `employees.json` absent | Exit 3 | Exit 3, file cannot be opened | Pass |
| 21 | Execution error | `SELECT name FROM students;` with a trailing comma in `students.json` | Exit 3 | Exit 3, invalid JSON reported with its position | Pass |

## 9.4 Automated Integration Tests

The integration tests in `dashboard/test/dashboard.test.mjs` run the real executable and the real server.

| Test | What is checked | Result |
|---|---|---|
| Projection, filter, sort, limit | Exact documents returned; 19 tokens; 10 symbols; stages `$match`, `$sort`, `$limit`, `$project`; all 7 phases complete | Pass |
| Second collection | Employee query on `department` returns the two engineers in salary order | Pass |
| Nested `AND` / `OR` | Parenthesised filter returns the right documents and emits `$and` | Pass |
| Empty result | A valid query matching nothing still succeeds with 0 documents | Pass |
| Syntax error | Phase 2 fails, phases 3 to 7 are skipped, no pipeline is produced | Pass |
| Lexical error | The error is assigned to phase 1 and all later phases are skipped | Pass |
| Semantic errors | Wrong type, unknown field and unknown collection all stop at phase 3 | Pass |
| Input handling | Input is always SQL, never a file path; empty, oversized and null-byte input rejected | Pass |
| HTTP API | Pages served; database matches `data/students.json`; bad JSON 400, other origin 403, wrong content type 415, source files 404 | Pass |

**Table 9.3:** Automated integration tests (9 of 9 pass in about 0.5 s)

## 9.5 Performance Evaluation

A complete seven-phase run averaged 50.1 ms for test case 1 and 52.7 ms for test case 2, over 100 runs each. A run that only prints the help text averaged 30.8 ms, so process start-up dominates. The compiler phases, reading the collection file and printing the output account for about 20 ms. Through the dashboard, a query takes roughly 35 to 130 ms from request to response.

Memory use is fixed: every structure has a static size, and the largest, the engine's two arrays of 32 documents, take about 213 KB.

## 9.6 Output Evidence

Figure 9.2 shows the generated pipeline and the execution results for test case 1 from the command line. The remaining figures show the dashboard: a successful query (Figure 9.3), a query stopped by a semantic error (Figure 9.4), and the stored collection (Figure 9.5).

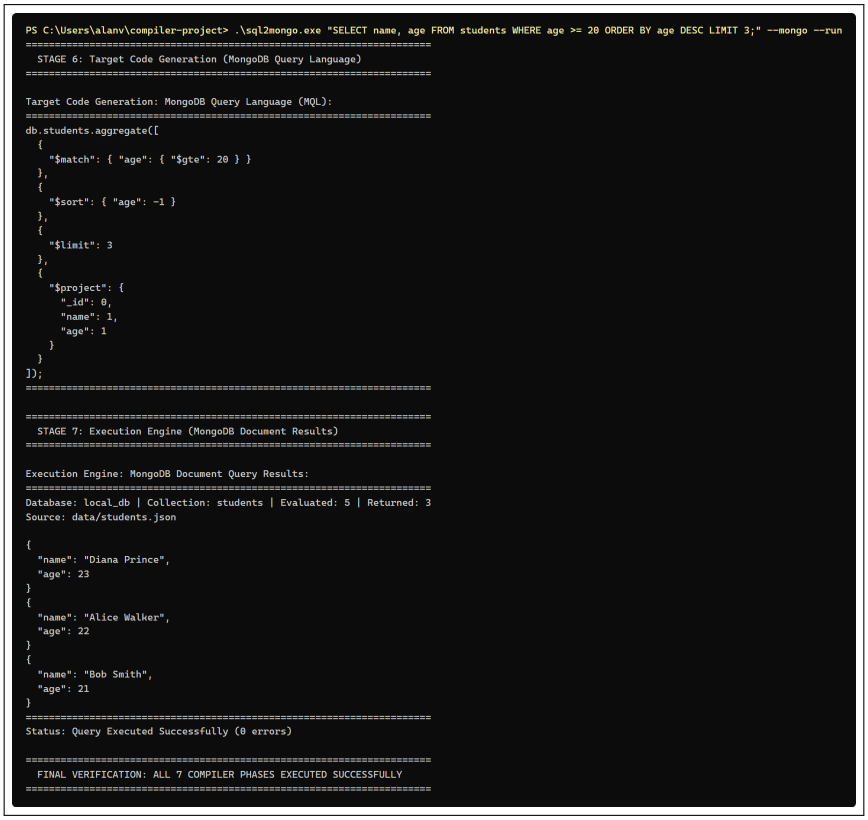



**Figure 9.2:** Test case 1 with `--mongo --run`: the pipeline, then three documents read from `data/students.json`

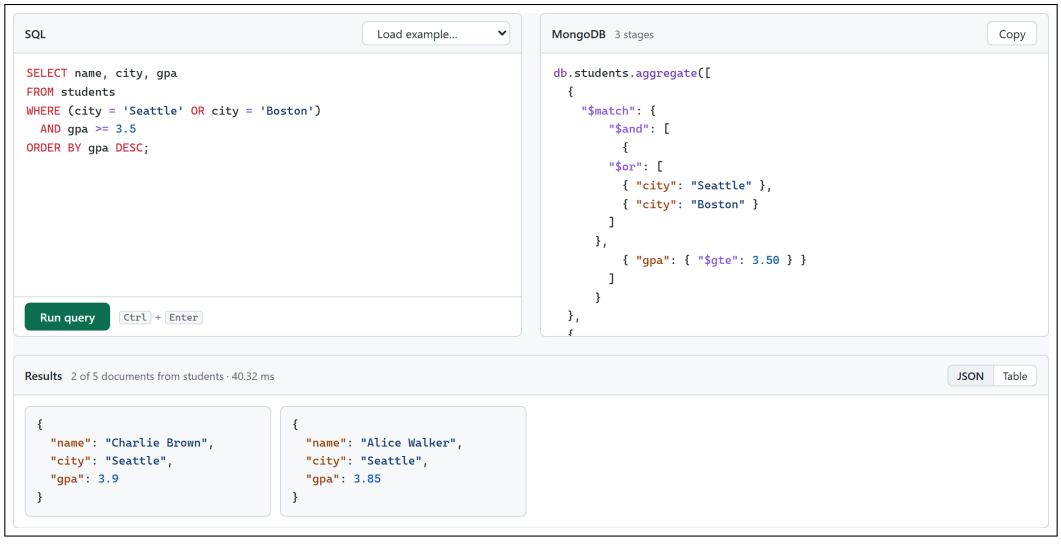



**Figure 9.3:** Query page: SQL with nested `AND`/`OR`, the generated pipeline and the two result documents

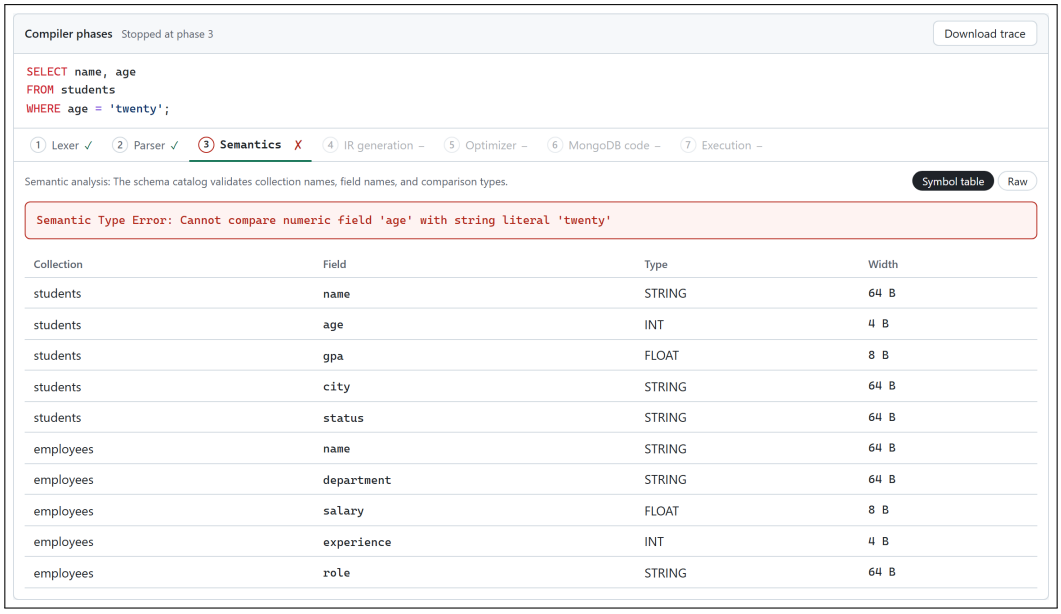



**Figure 9.4:** Compiler phases page for `WHERE age = 'twenty'`: phase 3 fails with the type error and phases 4 to 7 are not reached

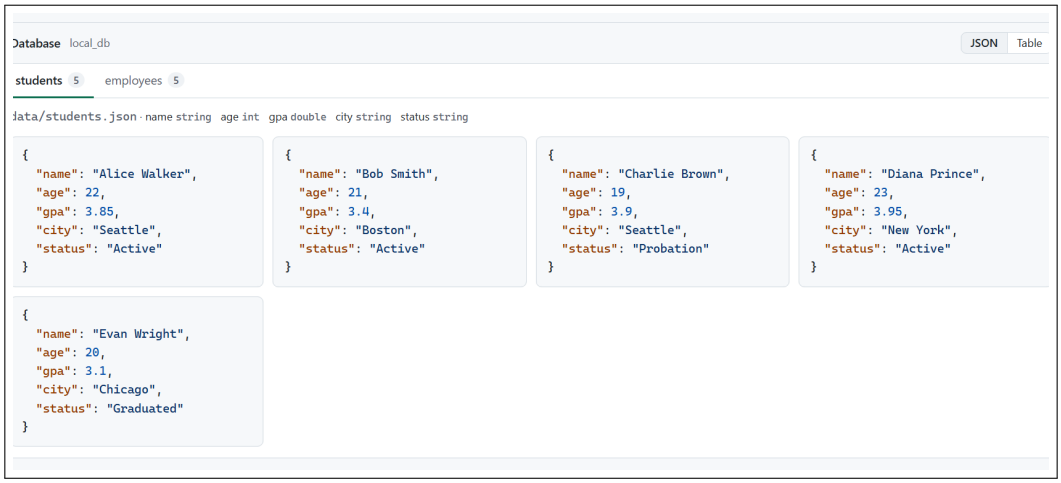



**Figure 9.5:** Database page: the `students` collection as stored in `data/students.json`

## 9.7 Findings

- **Case 7:** keywords and schema lookups are case-insensitive, but the generator copies identifiers as typed. MongoDB is case-sensitive, so `db.STUDENTS` and `"AGE"` would match nothing on a real server. Identifiers should be normalised to their catalog spelling.
- **Case 8:** MongoDB's `$limit` stage takes a positive integer [9], so `LIMIT 0` should be rejected during semantic analysis.

## 9.8 Summary of Results

Of 21 test queries, 19 behave exactly as expected and 2 are partial (cases 7 and 8); both partial cases run correctly in the engine but produce MQL that a real MongoDB server would treat differently. All 9 automated integration tests pass, and the compiler builds without warnings.

# Chapter 10: Conclusion

SQL2Mongo meets all nine objectives: it compiles a useful subset of SQL into valid MongoDB aggregation pipelines through seven hand-built phases, catches lexical, syntax and semantic errors before any code is generated, and checks each translation by running it on collections stored in the same JSON format MongoDB exports.

The project started from a practical problem I ran into during my internship: knowing SQL well, but having to write MongoDB queries. SQL2Mongo removes the need to translate each query by hand. Its phase-by-phase output, now also shown in the web dashboard, explains how each SQL clause maps to its MongoDB form, so it doubles as a learning aid for anyone making the same switch.

The project shows that the classic compiler pipeline carries over directly to query translation. Lexing, recursive-descent parsing, symbol tables, three-address code and code generation each have a clear job, and the AST holds them together. The same parsing technique was reused for the collection files. The visible output of every phase made the compiler easy to debug and is the core of its demonstration.

Testing with 21 queries and 9 automated tests confirmed correct translation and execution for valid input, and precise diagnostics for invalid input. It also found two gaps in target-code fidelity, identifier casing and `LIMIT 0`, which are the first items in Chapter 11.

# Chapter 11: Future Enhancements

1. **Target-code fidelity:** rewrite identifiers to their catalog spelling, and reject `LIMIT` values below 1.
2. **One schema source:** field types are declared in `schema_table.c` while the documents live in JSON. Reading the schema from a file in `data/`, or inferring it from the documents, would remove the duplication.
3. **Richer filters:** `NOT`, `IN` (`$in`), `LIKE` (`$regex`), `BETWEEN` (a `$gte`/`$lte` pair) and `OFFSET` (`$skip`).
4. **Aggregation and joins:** `GROUP BY` with `COUNT`, `SUM` and `AVG` (`$group`), and `JOIN` (`$lookup`).
5. **Real IR optimizations:** flatten chains such as `a AND b AND c` into one `$and` array, merge range conditions on the same field into one operator object, and drop redundant predicates.
6. **Error recovery:** panic-mode recovery in the parser, so one run can report several syntax errors instead of only the first.
7. **Larger collections:** replace the fixed limit of 32 documents with dynamically allocated storage, and support nested documents and arrays.

8. **Live validation:** run generated pipelines on a MongoDB server through the official C driver and compare the results with the built-in engine, using the same JSON files loaded with `mongoimport`.

# References


[1] A. V. Aho, M. S. Lam, R. Sethi and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Boston: Pearson Addison-Wesley, 2006.

[2] E. F. Codd, "A relational model of data for large shared data banks," *Communications of the ACM*, vol. 13, no. 6, pp. 377–387, 1970.

[3] P. G. Selinger, M. M. Astrahan, D. D. Chamberlin, R. A. Lorie and T. G. Price, "Access path selection in a relational database management system," in *Proc. ACM SIGMOD*, 1979, pp. 23–34.

[4] B. W. Kernighan and D. M. Ritchie, *The C Programming Language*, 2nd ed. Englewood Cliffs: Prentice Hall, 1988.

[5] T. Bray, Ed., "The JavaScript Object Notation (JSON) Data Interchange Format," RFC 8259, Internet Engineering Task Force, 2017. https://www.rfc-editor.org/rfc/rfc8259

[6] MongoDB, Inc., "SQL to Aggregation Mapping Chart," MongoDB Database Manual. https://www.mongodb.com/docs/manual/reference/sql-aggregation-comparison/

[7] MongoDB, Inc., "Aggregation Pipeline," MongoDB Database Manual. https://www.mongodb.com/docs/manual/core/aggregation-pipeline/

[8] MongoDB, Inc., "Aggregation Pipeline Optimization," MongoDB Database Manual. https://www.mongodb.com/docs/manual/core/aggregation-pipeline-optimization/

[9] MongoDB, Inc., "$limit (aggregation stage)," MongoDB Database Manual. https://www.mongodb.com/docs/manual/reference/operator/aggregation/limit/

[10] MongoDB, Inc., "mongoexport," MongoDB Database Tools documentation. https://www.mongodb.com/docs/database-tools/mongoexport/

[11] MongoDB, Inc., "mongotranslate SQL Translation Tool," MongoDB BI Connector documentation. https://www.mongodb.com/docs/bi-connector/current/reference/mongotranslate/

[12] V. Russell, "sql-to-mongo-db-query-converter," GitHub repository. https://github.com/vincentrussell/sql-to-mongo-db-query-converter

[13] *Compiler Design Laboratory Project Instruction Manual*, BCSE307P, Fall Semester 2026–27.


Web sources accessed 28 October 2026.

# Appendix A: User Instructions

## A.1 What You Need

- GCC (MinGW on Windows) on the `PATH`, to build the compiler.
- Node.js 20 or newer, for the dashboard and the tests. No `npm install` is needed.

## A.2 Build

From the `dashboard` folder:

```
npm run build:compiler
```

This runs `gcc -std=c11 -Wall -Wextra` on every file in `src/` and writes `sql2mongo.exe` to the project root.

## A.3 Start the Web Interface

```
cd dashboard
npm start
```

Open http://127.0.0.1:3000. Write a query or choose an example, then select **Run query** or press Ctrl+Enter. To use another port, set the `PORT` environment variable first.

## A.4 Command Line

Run on a query file or an inline query. With no flag, all phases run (`--all`).

```
sql2mongo.exe query.sql --all
sql2mongo.exe "SELECT name, age FROM students WHERE age >= 20 ORDER BY age DESC
    LIMIT 2;" --mongo
```

| Flag | Shows |
|---|---|
| `--tokens` | Token table (lexical analysis) |
| `--tree` | Concrete parse tree |
| `--ast` | Abstract syntax tree |
| `--schema` | Symbol table and semantic check result |
| `--tac`, `--quads`, `--triples` | Three-address code, quadruples, triples |
| `--opt` | Optimization report |
| `--mongo` | Generated MongoDB pipeline |
| `--run` | Query results from the execution engine |
| `--all` | Every phase in order (default) |

**Table A.1:** Command-line flags

### A.5 Data Files

Each collection is a JSON array of flat documents in `data/<collection>.json`. Values may be strings or numbers. A new field must also be added to the symbol table in `src/schema_table.c`, or queries that use it are rejected as semantic errors.

## Appendix B: Project Structure

```
compiler-project/
├── sql2mongo.exe        the compiled compiler
├── src/                 ten C modules (Table 8.2)
├── data/                collections: students.json, employees.json
├── dashboard/
│   ├── server.mjs       web server and API
│   ├── compiler.mjs     runs the compiler and parses its output
│   ├── build.mjs        builds sql2mongo.exe with GCC
│   ├── public/          index.html, app.js, styles.css
│   └── test/            integration tests
└── report/              this report (LaTeX source, figures and PDF)
```

The role of each C file is given in Table 5.1.

## Appendix C: Code Excerpts and Sample Data

### C.1 Operator Precedence in the Parser

From `sql_parser.c`. `OR` calls `AND`, which calls `primary`, so `AND` binds tighter. Each loop folds operands to the left.

```
static ASTFilter* parse_and_expr(Parser* p) {
    ASTFilter* left = parse_primary_filter(p);
    if (!left || p->has_error) return left;

    while (match(p, TOK_AND)) {
        ASTFilter* right = parse_primary_filter(p);
        if (!right || p->has_error) return left;
        left = ast_create_logical_filter(LOGIC_AND, left, right);
    }
    return left;
}

static ASTFilter* parse_or_expr(Parser* p) {
    ASTFilter* left = parse_and_expr(p);
    if (!left || p->has_error) return left;

    while (match(p, TOK_OR)) {
        ASTFilter* right = parse_and_expr(p);
        if (!right || p->has_error) return left;
        left = ast_create_logical_filter(LOGIC_OR, left, right);
    }
    return left;
}
```

## C.2 Sample Data

All test results in Chapter 9 can be checked against these two collections, stored in `data/students.json` and `data/employees.json`.

```
[
  { "name": "Alice Walker", "age": 22, "gpa": 3.85, "city": "Seattle", "status": "Active" },
  { "name": "Bob Smith", "age": 21, "gpa": 3.40, "city": "Boston", "status": "Active" },
  { "name": "Charlie Brown", "age": 19, "gpa": 3.90, "city": "Seattle", "status": "Probation" },
  { "name": "Diana Prince", "age": 23, "gpa": 3.95, "city": "New York", "status": "Active" },
  { "name": "Evan Wright", "age": 20, "gpa": 3.10, "city": "Chicago", "status": "Graduated" }
]
```

| name | department | salary | experience | role |
|---|---|---|---|---|
| Sarah Connor | Engineering | 95000 | 6 | Lead Dev |
| John Doe | Engineering | 72000 | 3 | Software Engineer |
| Jane Smith | Marketing | 65000 | 4 | Marketing Lead |
| Michael Scott | Management | 85000 | 10 | Regional Manager |
| Dwight Schrute | Sales | 58000 | 5 | Assistant Regional Manager |

**Table C.1:** The `employees` collection